

A Bayesian GAT-Informer Approach for Traffic Prediction

Mario Formato¹

¹Department of Computer, Control and Management Engineering, Sapienza University of Rome, 00185 Rome, Italy

Abstract

Traffic forecasting is a key component in the development of intelligent transportation systems, enabling optimized traffic management and route planning.

In this work, we address the problem of traffic forecasting by combining graph-based spatial encoding with transformer-based temporal modeling. Spatial relationships have been exploited using a Bayesian Graph Attention Network, which takes as input the adjacency matrix, that describes the connection between sensors within the road network and the measurements taken by the sensors. For the temporal analysis an Informer is used, a particular type of transformers, which main difference is placed in the attention mechanism, where only few queries are taken into account, the ones which scores exhibit the most variance.

The proposed architecture shows competitive results compared to the state of the art, maintaining stable results across different forecast horizon, making it suitable for real-time deployment.

Keywords

Transformer, Informer, Graph Attention Network, Traffic prediction, Time series Forecasting, Spatiotemporal modeling

1. Highlights

Some keypoints about my work are:

- Development of a proposed architecture for traffic prediction task;
- Introduction of a Bayesian component for uncertainty aware forecasting;
- Scalable and efficient temporal modeling using an Informer, which reduces the computational cost with respect to classic transformers;
- End-to-end learning framework capturing both spatial and temporal dependencies;
- Robustness across different prediction horizons, maintaining stability and accuracy over time.

2. Introduction

Traffic prediction is a crucial task in modern transportation system, in particular for urban mobility, road safety [1] and environmental sustainability. Accurate traffic forecasting improves traffic management, providing optimal route recommendation [2], reducing delays and fuel consumption, which also contributes to environmental sustainability, since traffic congestion leads to higher emissions of greenhouse gases and air pollutants[3] [4], mainly due to idling and stop and go driving.

The task of traffic prediction consists of forecasting traffic conditions based on historical data collected from sensors distributed at different points throughout the road network. The data used may represent different information

on traffic conditions, such as the mean speed of vehicles or the number of vehicles for unit of time, providing information about the general flow and drivability of the road network.

The prediction involves both temporal dimension and spatial location, which makes this task a spatio-temporal task. The spacial component of traffic is usually embedded in non-Euclidean domain, as the distance between two sensors cannot be measured with a Euclidean distance since the path between them follows the road and not a straight line. This type of space is often better represented as a graph [11], in which a node corresponds to a sensor and edges represent connections between two sensors, so basically if a real road connection between two sensors exists or not.

In order to accurately estimate the traffic condition, it is important to have a model capable of capturing both spatial and temporal patterns. Traditional time series models, such as recurrent neural networks (RNNs), struggle to learn complex interactions typical of road networks, since their main focus is on sequential dependencies and do not take into account non-Euclidean structure of transportation system.

Graph-based neural networks (GNNs) have emerged as powerful tools for traffic forecasting, as they can effectively model the complex spatial dependencies of a graph-like network.

Among Various different GNN architectures, graph attention networks (GATs) are particularly well-suited for this task [5]. Differently from classic GNNs, which aggregate information from neighboring nodes using fixed weights, GATs use attention mechanism to assign different important scores to each neighboring node. This enable the network to effectively distinguish the varying impact of different edges, allowing it to be more adaptable and

formato.2152143@studenti.uniroma1.it (M. Formato)



© 2025 Author. Please fill in the copyright clause macro

CEUR Workshop Proceedings (CEUR-WS.org)

appropriate for the dynamic nature of traffic.

With the spatial patterns extracted by the GAT, to capture both spatial and temporal dependencies, the architecture is further extended by integrating a transformer. In order to reduce the computational cost, an Informer [9] model has been chosen. Unlike traditional transformers, which suffer from quadratic complexity due to self-attention mechanism, informers use a sparse self-attention mechanism, which selects only the most important queries, basically the ones which have the highest variance in the attention score. This optimization reduces the time complexity from $O(N^2)$ to $O(N\log N)$.

Another important aspect in traffic prediction is the inherent uncertainty given by elements such as weather or accidents [6]. To account for this uncertainty, a Bayesian GAT (BGAT) is used instead of a traditional GAT. Differently from a standard GAT, which provides deterministic outputs, a BGAT learns both means and variances of the predictions, which allows the model to capture the variability in the data, which leads to a more robust model and more reliable and adaptive predictions.

3. Related works

Different methods have been exploited to solve traffic prediction task. First approaches used traditional statistical and machine learning approaches, such as Autoregressive Integrated Moving Average (ARIMA) or Support Vector Machine (SVM), which struggled to capture spatio-temporal patterns [15].

The primarily focus was on incorporating spatial information into models, using graph structures, to reflect the non-Euclidian space characteristics. Some examples are the Diffusion Convolutional Recurrent Neural Network (DCRNN) [10], in 2018, which added a graph convolution in a Recurrent Neural Network, or the work of Yu et al Spatio-Temporal Graph Convolutional Network (STGCN) [11], in 2018, in which a graph convolutional network was used. Transformers became widely used for traffic prediction task, given their ability of modeling temporal dependencies within data, some example are Traffic Transformer [12], in 2020, and STAEformer (STAE) [13], in 2023, which introduces a module capable of capturing spatio-temporal dependencies, using adaptive temporal embeddings and adaptive spatial embeddings, which are then combined and used as input to a Transformer.

4. Dataset

The dataset used for this work is METR-LA. This dataset was collected by loop detectors, installed on the Los Angeles Highway, between March and June 2012.

It consists of measurements of 207 sensors, which positions are shown in Figure 1, that have been taken every

5 minutes over a four months period. Each measurement represents the average speed of vehicles within a specific time interval.

The spatial relationship among sensors is encoded in an adjacency matrix, a matrix of 207×207 , in which each element A_{ij} encodes the strength of the connection, if one exists. Basically, if the element A_{ij} is equal to 0, there is no direct connection between the sensors i and j , otherwise, a non-zero value indicates a direct connection, which value captures how strongly the two sensors are spatially related.

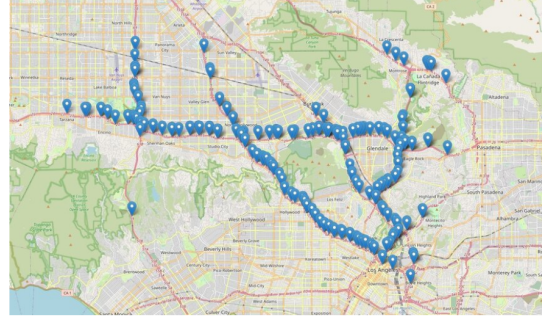


Figure 1: Geographical map of Los Angeles showing the locations of all traffic sensors used to collect the METR-LA dataset.

5. Preliminary

5.1. Graph Neural Networks

GNNs are a class of deep learning models designed to work with graph-structured data. Given a graph $G = \langle V, E \rangle$, in which V is the set of vertices, or nodes, and E the set of edges, each node represents an entity and edges capture relation between them.

At each layer, a node updates its representation by combining information from neighbors, usually through a message-passing mechanism. This mechanism consists of three steps: message computation, in which each node compute a message to send to its neighbors

$$m_{vu}^{(l)} = \text{Message}(h_v^{(l)}, h_u^{(l)}, e_{uv}) \quad (1)$$

based on the feature vector of u $h_u^{(l)}$ and v $h_v^{(l)}$ and the edge feature e_{uv} between u and v , if one exists; aggregation, the node aggregates all the incoming messages from its neighbors

$$m_v^{(l)} = \text{Aggregate}(m_{vu}^{(l)} | u \in N(v)) \quad (2)$$

in which $N(v)$ represents the set of neighbors of node v ; update, the node combines its current features with the

aggregate message, which produces a new representation

$$h_v^{(l+1)} = \text{Update}(h_v^{(l)}, m_v^{(l)}) \quad (3)$$

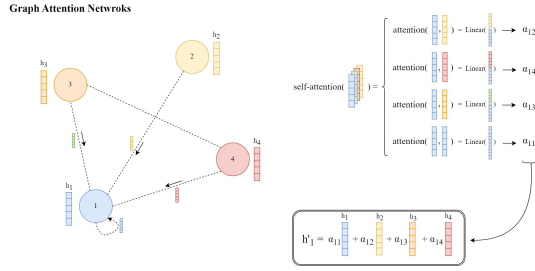


Figure 2: Working flow of GAT's attention mechanism

5.2. Graph Attention Network

Graph Attention Networks (GATs) are a variant of GNNs that introduce the attention mechanism, shown in Figure 2. This mechanism enables the network to give different importance to each neighbor during the aggregation phase of the message passing mechanism. This mechanism allows the model to make more accurate predictions by focusing on the most relevant information. Each node has a features vector $h_i \in R^F$. A learnable weight matrix $W \in R^{F' \times F}$ is used to linearly transform the input features

$$h'_i = Wh_i \quad (4)$$

For each edge (i, j) , the attention coefficient is computed. First, one should evaluate the attention score e_{ij} , which reflects the importance of node j to node i . This term is usually computed as:

$$e_{ij} = \text{LeakyReLU}(a^T [h'_i || h'_j]) \quad (5)$$

with $a \in R^{2F'}$.

To make coefficient comparable across different neighbors, the attention scores are normalized using the softmax function, obtaining the attention coefficients

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in N(i)} \exp(e_{ik})} \quad (6)$$

After computing the attention coefficients it is possible to proceed with the aggregation phase. The output feature vector for each node is a weighted sum of its neighbors features

$$h''_i = \sigma \left(\sum_{j \in N(i)} \alpha_{ij} h'_j \right) \quad (7)$$

in which σ is an activation function.

5.3. Transformer

Transformers are a class of deep learning model initially designed for tasks of natural language processing, to become a highly used architecture for modeling sequential data, given their ability to capture long-range dependencies.

The main difference from architectures like RNN and Long Short-Term Memory (LSTM), Transformers rely on the self-attention mechanism, which makes it possible to process the entire sequence in parallel, capturing relations between all the elements in the sequence.

Given an input vector $X = [x_1, x_2, \dots, x_n]$, the self-attention mechanism computes three vectors: query (Q), key (K) and values (V). This computation is done using three different matrices, respectively W^Q , W^K and W^V , which are learnable parameters.

$$Q = XW^Q, K = XW^K, V = XW^V \quad (8)$$

The attention is computed then with the following equation:

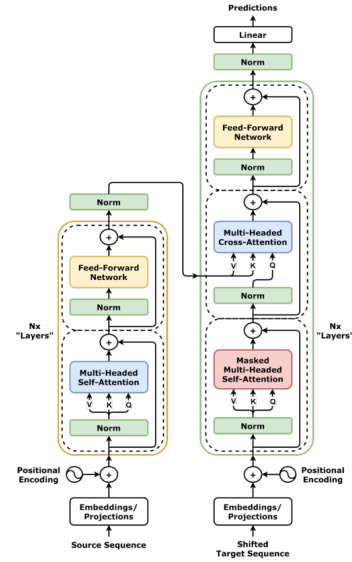


Figure 3: Transformer architecture with multi-head self-attention mechanism

tions:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (9)$$

in which d_k represents the dimension of the keys, used to scale the product.

The original architecture consists of two main components: encoder and decoder. The encoder takes the sequential input and extracts a representation of each element. It consists of different layers, each one composed

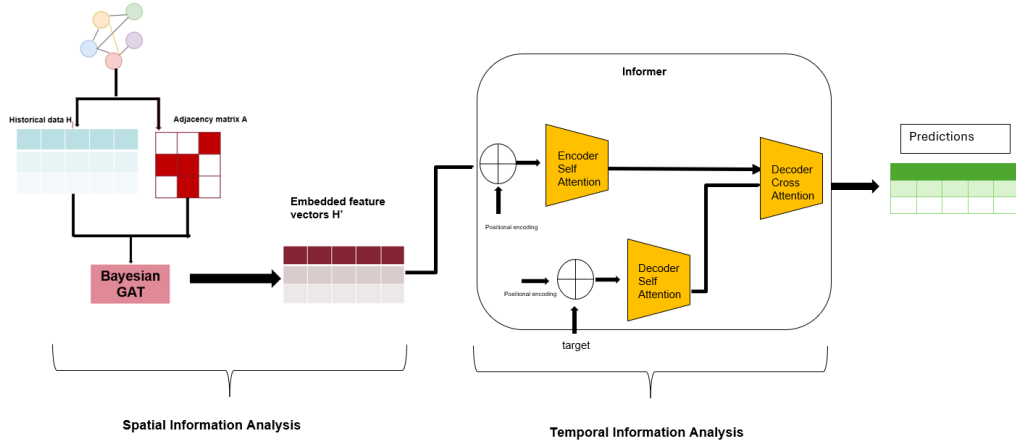


Figure 4: Overview of the proposed BGAT-Informer architecture. Here we can distinguish two modules. The first one for the spatial analysis with the BGAT, which takes the input window of measurements and the adjacency matrix and output the embedded features vectors. The second module is for temporal analysis, takes as input the output of the first module and outputs the prediction for the traffic forecasting task.

of two components: a multi-head self-attention mechanism and a fully connected network.

The multi-head self-attention mechanism allows learning attention in different subspaces by splitting query, key and value vectors into multiple heads and performing in parallel, then simply concatenate the results and multiply by a learned output projection matrix W^0

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^0 \quad (10)$$

. The output of the multi-head self-attention mechanism is normalized, passed through a fully connected network and normalize again.

The decoder is primarily responsible for the generation of the output sequence based on the representation of the input given by the encoder.

In the official architecture it consists of six layers, each one defined by three sub-layers.

The first sub-layer takes as input the target sequence, passed through a normalization layer and a masked Multi-Head Self-Attention mechanism, similar to the self-attention of the encoder, with the only difference that the input is masked to prevent attending future positions in the sequence. Basically, to predict the element in position t , only the elements of position smaller than t are considered, the others are masked, which is done by setting the attention score to $-\infty$.

The second sub-layer contains a Multi Head Cross-Attention mechanism, which takes as input the encoder output and uses the output of the Masked Self-Attention as the query.

The last sub-layer is a feed-forward network.

5.4. Informer

Traditional Transformer-based models have achieved impressive performance in sequence modeling tasks. However, their application to long sequences forecasting is limited because of the computational complexity of the self-attention mechanism, which is quadratic with respect to the input length.

To overcome this problem, Informer [9] was introduced, a Transformer variant specifically designed for efficient long-term time series forecasting, capable of achieving a good trade-off between accuracy and computational efficiency in terms of time and memory usage.

The innovation in the Informer lies in the probabilistic sparse self-attention mechanism. Basically, instead of computing the full self-attention matrix, the informer only considers the queries which scores exhibit the greatest variance, which are the ones with the highest impact, with the assumption that they carry out the most informative signals.

The attention is computed as:

$$A(Q, K, V) = softmax(\frac{\bar{Q}K^T}{\sqrt{d_k}})V \quad (11)$$

where $\bar{Q}$ are the top u queries, where $u = c * \ln L_Q$, with c a constant factor.

This innovation allows to reduce the complexity from $O(N^2)$ to $O(N \log N)$, making it more scalable for long sequence tasks.

Model	BGAT Layers	Encoder Layers	Decoder Layers	Validation Loss
M_1	1	2	2	6.1019
M_2	1	3	3	5.8288
M_3	2	4	4	5.6303
M_4	2	6	6	5.6064
M_5	2	8	8	5.6190
M_6	3	4	4	5.6347

Table 1
Model configuration and Validation Loss after 100 epochs.

6. Methods

6.1. Problem definition

The objective of the traffic prediction task is to predict future traffic conditions based on historical observations taken by sensors distributed all over the road network. Taking a graph $G = \langle V, E \rangle$, V is the set of vertices, which are basically the set of sensors, while E is the set of edges, representing a physical connection between two sensors on the network.

To each node $v_i \in V$ is associated a multivariate time series $X_i = \{x_i^t\}_{t=1}^T$, where $x_i^t \in R^d$ contains the d measurements taken by the sensors i at time step t . For example, in the case of the METR-LA dataset, the only measurement is the mean speed, so $d=1$.

The goal is to forecast the future values $\{x_i^t, \dots, x_i^{t+H}\}_{i=1}^N$ for each node i , given the previous L observations. Basically the model learns a mapping:

$$f : \{X_i^{(t-L+1:t)}\}_{i=1}^N \rightarrow \{X_i^{(t+1:t+H)}\}_{i=1}^N \quad (12)$$

This problem is a spatio-temporal task, since it involves: temporal dependencies, due to sequential nature of traffic data, and spatial dependencies, since traffic conditions are influenced by the conditions of neighboring areas.

6.2. Spatial analysis: Bayesian Graph Attention Network

The first step of the analysis focuses on modeling spatial dependencies present in traffic data. Spatial relationships are encoded in the topology of the road network, where each sensor is represented as a node within a traffic graph, and physical road connections define the edges between nodes.

To effectively capture these dependencies, a BGAT has been deployed, an extension of a classic GAT, which incorporates uncertainty estimation through Bayesian techniques, instead of producing deterministic node embeddings as classical GATs, which can be limiting in real-world scenarios.

Introducing uncertainty into the model allows it to express a level of confidence in its predictions, rather than

relying on single-value outputs. This is particularly important for task like traffic prediction, where data may be noisy, incomplete, or lack critical contextual information. Dataset METR-LA only takes into account data about mean speed in a slot of five minutes, while other important information may be useful to correctly address this task such as weather events, road incidents, or construction works, since they significantly influence traffic dynamics. A deterministic model may become overconfident in such situations, leading to unreliable outputs.

The BGAT addresses this issue by modeling each node representation as a probability distribution. The module is composed of several layers, each of them consists of: a GAT layer, which aggregates information from neighboring nodes using attention mechanism, and a learned log-variance parameter for uncertainty estimation, shared across all nodes, used to define the standard deviation of the output distribution.

Specifically, the GAT layer outputs a mean of a Gaussian distribution, while the standard deviation is derived from the learned log-variance. During both training and inference, node representations are sampled from these distributions, enabling the model to capture uncertainty reflecting the lack of important information.

The presence of different layers allows to build a hierarchical spatial representation, allowing it to capture both local and higher-order spatial dependencies. The final output is a node-wise latent representations, passed to the temporal modeling block for sequence prediction.

6.3. Temporal analysis: Informer

After the spatial dependencies have been modeled using a BGAT, which returns a temporal ordered sequence of spatially-informed traffic states for each node, it is crucial to capture the temporal dynamics of the traffic over time, since future states are heavily influenced by the historical data.

To address this aspect, an Informer architecture is chosen. This choice was motivated by the need to handle relatively long input sequences for a large number of sensors, without suffering from high memory and computational cost associated with standard Transformers architectures. This limitation becomes particularly relevant when aim-

Model	15 min		30 min		60 min	
	MAE	RMSE	MAE	RMSE	MAE	RMSE
SVR	3.99	8.45	5.05	10.87	6.72	13.76
ARIMA	3.99	8.21	5.15	10.45	6.90	13.23
DCRN	2.77	5.78	3.15	6.45	3.60	7.59
STGCN	2.88	5.74	3.47	7.24	4.59	9.4
STAEformer	2.65	5.11	2.97	6.00	3.34	7.02
ASTGCN	4.86	9.27	5.43	10.61	6.51	12.52
T-Graphomer	2.62	4.93	2.90	5.53	3.19	6.12
BGAT-Informer	3.22	7.76	3.66	9.01	4.57	10.87

Table 2
Spatiotemporal Traffic forecasting results on Metr-LA dataset.

ing for real-time inference in traffic forecasting applications.

To address this problem the informer architecture employs a ProbSparse self-attention mechanism, which selects only the most relevant query-key pairs, significantly reducing the complexity, this mechanism allows the model to scale to longer time horizons without incurring excessive computational costs without sacrificing accuracy.

In this architecture, each ordered sequence of spatially enriched feature vector of the last 12 time steps is used as input to the encoder. It processes these sequences to capture both short-term fluctuations and longer-term patterns.

During inference, the self-attention mechanism of the decoder takes as input a sequence of 12 time steps, where each element is a copy of the last known observation from the input window. The the decoder perform cross-attention with encoder output as keys and values and the decoder’s self attention mechanism’s output as queries, which allows each predicted time step to attend to relevant moments in the input sequence.

The output of the decoder represents the predicted traffic states for the next 12 time steps, reflecting complex traffic patterns and dependencies, and leading to accurate and robust forecasting.

7. Experiments and results

7.1. Experiments

7.1.1. Pre-processing

The dataset is split in the subsets: 70% for the training, 15% for validation and 15% for testing. From the raw data, sequences of fixed length have been extracted for defining both input and prediction windows; during the training, the input and prediction windows used for training and validation are set to a length of 12.

To prepare the data, a Standard Scaler is used, in order to normalize the input features, ensuring a zero mean

and a variance of one. The normalization is essential to stabilize the training process and improving model convergence.

7.1.2. Implementation Details: Model Architectures

For this architecture, different possible model configurations have been explored, varying the combinations of layers’ numbers for the BGAT and the encoder/decoder of the Informer.

Each variant has been trained for 100 epochs in which the training dynamics and the validation performance have been monitored. This process enabled to understand which combination was the most promising one. In table 1 there is the list of all the configuration selected for the training phase. Based on the results obtained from the different configurations, the best-performing ones appears to be the model M_4 .

7.2. Training details

The training process involves optimizing the parameters of both the BGAT and the Informer, in an end-to-end manner, using Adam optimizer. Adam has been chosen because of its robustness and its ability to adapt the step size for each parameter throughout the training process, which is particularly beneficial for deep architectures. The learning rate, set to 10^{-4} , and batch size of 128, were chosen empirically to ensure stable convergence, avoiding oscillations and vanishing gradients.

During training, the goal was to minimize the Mean Absolute Error (MAE) on predicted future time steps given an historical window of measurements. In particular, the historical window consists of twelve measurements (which is a window of an hour) and the prediction consists of the successive twelve measurements.

Two different techniques have been used during training, both affecting the input of decoder’s self attention mechanism. The first one is teacher forcing, the ground truth future sequence is given as input to the decoder, which at-

tends over the actual future values during self-attention. This approach helps model to converge faster and learn more accurate temporal dependencies, especially in the early stages of training. The second approach, the one applied during inference, consists in using a sequence of twelve elements, which correspond to the last observation in the input sequences. This simple strategy proved empirically to lead to better performance with respect to other more complex approaches, as full autoregressive decoding.

To leverage the benefits of both approaches, the model M_4 was trained for 400 epochs: the first 300 epochs using teacher forcing, which helps the model to learn temporal dynamics with strong supervision, while the remaining 100 epochs were carried out by using the second approach, to gradually align with the inference set up.

Additionally, dropout regularization was applied in both modules, in order to reduce overfitting. The dropout rate chosen is 0.1.

The full training process, for model M_4 required around 4 hours, using a single NVIDIA Tesla P100 GPU provided by the Kaggle environment.

7.3. Results

To evaluate the performance of the proposed architecture on the METR-LA dataset, two different loss metrics have been considered: MAE and Root Mean Squared Error (RMSE), both expressed in mph, MAE offers an interpretable measure of the average prediction error, while RMSE emphasizes larger deviations, making it more sensitive to outliers and useful for assessing robustness.

The evaluation has been conducted across multiple prediction horizons, specifically with target windows of 3, 6 and 12 time steps, which correspond to windows of 15 minutes, 30 minutes and one hour respectively. These different prediction horizons help analyze the model performance under increasing temporal uncertainty, which is essential in traffic forecasting, since both short-term responsiveness and longer-term planning matter.

The results are summarized in table 2. For example, for a window of 15 minutes the model achieves a MAE of 3.22 mph, a sufficiently small margin in urban traffic, that indicates that the model is accurate enough for applications such as congestion detection or route recommendation system.

Not surprisingly, both MAE and RMSE increase with the length of the forecast horizon, due to the growing uncertainty and complexity for longer-term predictions, where minor errors can accumulate over time. However, the BGAT-Informer exhibits a smooth degradation in accuracy, indicating stable generalization even in the longer range. Even at a one-hour horizon, where MAE increases at 4.57 mph, the predictions remain within an

acceptable range, which confirms the meaningful accuracy over longer time predictions.

Although the BGAT-Informer does not outperform the most accurate models, like T-Graphormer or STAEformer, it achieves a solid trade-off between accuracy and computational efficiency, which translates into practical advantages for real-world deployment.

BGAT-Informer adopts a non-autoregressive decoding approach, which reduces inference time, since the decoder input remains fixed and does not depend on previous predictions, which makes it possible to generate all the predictions in parallel in a single forward pass.

Moreover, the Informer backbone contributes to efficiency through its ProbSparse attention mechanism, it reduces the computational complexity of the attention mechanism, which allows the model to handle longer historical input sequences with significantly lower memory and computation costs compared to standard Transformers. This is really important in real-time traffic forecasting, where new data arrives continuously from a large number of sensors and predictions must be generated quickly under hardware constraints.

As previously stated, training for BGAT-Informer on METR-LA required about 4 hours on a single NVIDIA Tesla P100 GPU. For comparison T-Graphormer took approximately 4.5 hours to train on a high-performance setup with 4 NVIDIA V100 GPUs. This suggests that BGAT-Informer offers competitive training times even with less powerful hardware, further supported by its non-autoregressive decoding strategy and ProbSparse attention.

8. Conclusion

In this work it has been presented a forecasting model based on hybrid architecture that combines Graph Attention Network and a Transformer Informer, to address the problem of multivariate time series prediction. This architecture leverages both spatial and temporal dependencies.

The experiments have been conducted on the dataset METR-LA which contains measurements about mean speed at different points of Los Angeles road network. These experiments demonstrate that the network is capable of reaching quite good results, showing promising performance and stability. This suggests that this approach could be a good baseline for further enhancements. For future works I may investigate more advanced decoder techniques or considering training on larger dataset, which includes important features such as weather or special events.

9. Acknowledgments

This study has been conducted under the supervision of Prof. Christian Napoli and Tutor Assistant Valerio Ponzi. The project has been developed within the Elective in Artificial Intelligence (EAI) course, offered by La Sapienza University of Rome, as part of my Master's program in Artificial Intelligence and Robotics.

References

- [1] Anil Kumar Jakkani et al "Enhancing Road Safety Through Ai: A Comprehensive Analysis of Accident Prevention Strategies" <https://www.researchgate.net/publication/38327305>
- [2] Aristeidis Mystakidis et al "Advances in Traffic Congestion Prediction: An Overview of Emerging Techniques and Methods" <https://www.mdpi.com/2624-6511/8/1/25>
- [3] Florian Kellner "Exploring the impact of traffic congestion on CO2 emissions in freight distribution networks" <https://link.springer.com/article/10.1007/s12159-016-0148-5>
- [4] Hang Ding et al. "Quantifying the impact of built environment on traffic congestion: A nonlinear analysis and optimization strategy for sustainable urban planning" <https://www.researchgate.net/publication/389277620>
- [5] Park et al. "ST-GRAT: A Novel Spatio-temporal Graph Attention Network for Accurately Forecasting Dynamically Changing Road Speed" <https://arxiv.org/pdf/1911.13181>
- [6] Sengupta et al. "A Bayesian approach to quantifying uncertainties and improving generalizability in traffic prediction models" <https://arxiv.org/pdf/2307.05946>
- [7] Vaswani et al. "Attention is all you need": <https://arxiv.org/pdf/1706.03762>
- [8] Velickovi et al. "Graph Attention Networks": <https://arxiv.org/pdf/1710.10903>
- [9] Zhou et al. "Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting": <https://arxiv.org/abs/2012.07436>
- [10] Yaguang et al. "Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting": <https://arxiv.org/pdf/1707.01926>
- [11] Yu et al. "Spatio-Temporal Graph Convolutional Networks: A Deep Learning Framework for Traffic Forecasting": <https://arxiv.org/pdf/1709.04875>
- [12] Ling Cai et al. "Traffic transformer: Capturing the continuity and periodicity of time series for traffic forecasting": <https://onlinelibrary.wiley.com/doi/epdf/10.1111/tgis.12644>
- [13] Hangchen Liu et al. "STAEformer: Spatio-Temporal Adaptive Embedding Makes Vanilla Transformer SOTA for Traffic Forecasting": <https://arxiv.org/pdf/2308.10425v5>
- [14] Hao Yuan Bai and Xue Liu "T-Graphormer: Using Transformers for Spatiotemporal Forecasting": <https://arxiv.org/pdf/2501.13274>
- [15] Shengnan Guo et al. "Attention Based Spatial-Temporal Graph Convolutional Networks for Traffic Flow Forecasting": <https://guoshnbjtu.github.io/pdfs/AAAI2019-GuoS.2690.pdf>